

Clase 4 - Temario

- Funciones Lambda
- Map, filter, reduce



FUNCIONES LAMBDA

- SE REFIERE A UNA **PEQUEÑA FUNCIÓN ANÓNIMA**, YA QUE CARECEN DE NOMBRE
- **NO SE DEFINEN CON LA PALABRA CLAVE DEF**
- SE DEFINEN EN UNA SOLA LÍNEA QUE EJECUTA UNA SOLA EXPRESIÓN
- PUEDEN TENER CUALQUIER NÚMERO DE ARGUMENTOS, PERO PUEDEN TENER UNA **EXPRESIÓN**
- **TODA FUNCIÓN LAMBDA SE PUEDE CONVERTIR EN UNA FUNCIÓN NORMAL, PERO NO VICEVERSA**

¿CUANDO USAR FUNCIONES LAMBDA?

- SE USAN CUANDO NECESITAMOS CREAR FUNCIONES **PEQUEÑAS, SIMPLES Y RÁPIDAS**, SIN NOMBRE Y QUE SE USAN EN EL MOMENTO
- CUANDO LA LÓGICA ES CORTA, EN LO GENERAL UNA SOLA EXPRESIÓN
- CUANDO LA FUNCIÓN SE USA UNA SOLA VEZ



¿CUANDO NO USAR FUNCIONES LAMBDA?

- CUANDO LA FUNCIÓN ES COMPLEJA
- CUANDO TIENE MUCHAS LÍNEAS
- CUANDO NECESITÁS REUTILIZARLA
- CUANDO QUERES QUE EL CÓDIGO SEA MÁS LEGIBLE



FUNCIONES LAMBDA

lambda p1,p2,...,pn: expresión



argumentos

FUNCIÓN NORMAL

```
def doble(numero):  
    return numero*2
```

FUNCIÓN LAMBDA

```
lambda numero :numero*2
```

Para utilizarla se debe guardar en una variable y utilizarla como haríamos con una función normal

```
doble=lambda numero :numero*2  
print(doble(88))
```

FUNCIONES LAMBDA

Comprobar si un número es impar

```
impar = lambda numero : numero%2!=0
```

Invertir una cadena utilizando rebanado:

```
invertir = lambda cadena: cadena[::-1]
```

Podemos utilizar varios argumentos, por ejemplo, para sumar dos números:

```
sumar = lambda numero1,numero2: numero1+numero2
```

FUNCIONES LAMBDA

Calcular la potencia de un número:

```
potencia = lambda base, exponente : base ** exponente
```

Las funciones lambda en Python se pueden invocar inmediatamente de su definición utilizando paréntesis para pasar argumentos. **Solo se va a poder utilizar con estos argumentos**

Esto se conoce como función lambda inline

Sumar tres números

```
resultado = (lambda x,y,z: x+y+z)(1,2,3)
```

FUNCIONES LAMBDA - EJEMPLOS

Concatenación de cadenas

```
resultado = (lambda a,b: a + b) ("Hola, ", "mundo")
```

Comparación de dos números

```
resultado = (lambda x,y: x if x>y else y)(7,3)
```

Elevar un número a otro número

```
resultado = (lambda base,exp: base**exp)(2,3)
```

Funciones Lambda con cláusula else

lambda p1,...,pn: {valor condicion verdadera} if condicion else {valor condicion falsa}

```
cuadradoSiesPositivo = lambda numero : numero ** 2 if numero>0 else 0

print(cuadradoSiesPositivo(-1))
print(cuadradoSiesPositivo(2))
```

Aunque se puede utilizar else en una expresion lambda, si la lógica es demasiado compleja, puede ser más legible utilizar una función regular en su lugar.

FUNCIÓN MAP

- Por medio de la función `map()` podemos aplicar una función a cada uno de los elementos de una lista y obtener un conjunto de datos ya transformados

`map(funcion, lista)`

FUNCIÓN MAP

FUNCIÓN NORMAL

```
enteros=[1,2,4,7]
cuadrados=[]

for numero in enteros:
    cuadrados.append(numero**2)

print(cuadrados)#1,4,16,49
```

FUNCIÓN MAP

```
enteros=[1,2,4,7]
cuadrados=list(map(lambda numero:numero**2,enteros))

print(cuadrados)
```

La función map() retorna un objeto del tipo map().

Para que podamos trabajar con dicho resultado, es que lo combinamos con el uso de la función list()

FUNCIÓN MAP

MAP CON UNA FUNCIÓN NORMAL

```
1 def elevarCuadrado(numero):  
2     return numero**2  
3  
4  
5 numeros=[1,3,5,8]  
6  
7 resultado=list(map(elevarCuadrado,numeros))  
8  
9 print(resultado)
```

FUNCIÓN Filter

- La función filter() retorna un conjunto de elementos que cumplen una determinada condición (True)

filter(funcion, lista)

- La función filter retorna un objeto filter. Para poder utilizarlo, es necesario convertirlo en una lista con la función list.
- La función que se utiliza siempre debe devolver un bool

Función filter

Obtener los pares de una lista

FUNCIÓN NORMAL

```
valores=[1,2,3,4,5,6,7,8,9]
pares=[]

for valor in valores:
    if valor%2==0:
        pares.append(valor)

print(pares)
```

FUNCIÓN FILTER CON LAMBDA

```
valores=[1,2,3,4,5,6,7,8,9]
pares=list(filter(lambda numero: numero%2==0, valores))
print(pares)
```

Función filter

Función filter con una función que no es lambda

```
def mayorEdad(edad):  
    return edad >= 18  
  
edades = [11, 15, 19, 21, 5]  
  
edadesMayores = list(filter(mayorEdad, edades))  
  
print(edadesMayores)
```

FUNCIÓN REDUCE

- Toma como argumento un conjunto de valores (lista, tupla o cualquier elemento iterable) y lo “reduce” a un único valor

reduce(funcion, lista)

- Pertenece al módulo functools.
- La función que se pasa como primer parámetro siempre debe tener dos parámetros

FUNCIÓN REDUCE

- Reduce() se encargará de llamarla acumulativamente (es decir preservando el resultado de llamadas anteriores) de izquierda a derecha.

FUNCIÓN REDUCE

```
from functools import reduce

def suma(a,b):
    return a+b

print(reduce(suma, [1,2,3,4]))
```

Es similar a hacer:

```
def suma(a,b):
    return a+b

print(suma(suma(suma(1,2),3),4))
```

Autoevaluación



Guía de ejercicios Listas



Ejercicio 1

Crear una función lambda que reciba dos números y devuelva el mayor



Ejercicio 2

Realizar el promedio de tres números



Ejercicio 3

Determinar si una palabra empieza con una vocal

Guía de ejercicios Listas



Ejercicio 1

```
mayor = lambda a, b: a if a > b else b  
print(mayor(3, 7))
```



Ejercicio 2

```
prom = lambda a, b, c: (a + b + c) / 3  
print(prom(4, 5, 6))
```



Ejercicio 3

```
vocal = lambda s: s[0].lower() in "aeiou"  
print(vocal("Ana"))
```